JavaScript is disabled on your browser.



compact1, compact2, compact3

java.nio.channels

# Interface ReadableByteChannel

**All Superinterfaces:**

AutoCloseable, Channel, Closeable

**All Known Subinterfaces:**

ByteChannel, ScatteringByteChannel, SeekableByteChannel

**All Known Implementing Classes:**

DatagramChannel, FileChannel, Pipe.SourceChannel, SocketChannel

---

```
public interface ReadableByteChannel
extends Channel
```

A channel that can read bytes.

Only one read operation upon a readable channel may be in progress at any given time. If one thread initiates a read operation upon a channel then any other thread that attempts to initiate another read operation will block until the first operation is complete. Whether or not other kinds of I/O operations may proceed concurrently with a read operation depends upon the type of the channel.

**Since:**

1.4

## *Method Summary*

**Abstract Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| int | read(ByteBuffer dst) | |

| | | |
|---|---|---|
| | | Reads a sequence of bytes from this channel into the given buffer. |

### Methods inherited from interface java.nio.channels.Channel

`close`, `isOpen`

## *Method Detail*

### read

```
int read(ByteBuffer dst)
  throws IOException
```

Reads a sequence of bytes from this channel into the given buffer.

An attempt is made to read up to *r* bytes from the channel, where *r* is the number of bytes remaining in the buffer, that is, `dst.remaining()`, at the moment this method is invoked.

Suppose that a byte sequence of length *n* is read, where `0` <= *n* <= *r*. This byte sequence will be transferred into the buffer so that the first byte in the sequence is at index *p* and the last byte is at index *p* + *n* `-` `1`, where *p* is the buffer's position at the moment this method is invoked. Upon return the buffer's position will be equal to *p* + *n*; its limit will not have changed.

A read operation might not fill the buffer, and in fact it might not read any bytes at all. Whether or not it does so depends upon the nature and state of the channel. A socket channel in non-blocking mode, for example, cannot read any more bytes than are immediately available from the socket's input buffer; similarly, a file channel cannot read any more bytes than remain in the file. It is guaranteed, however, that if a channel is in blocking mode and there is at least one byte remaining in the buffer then this method will block until at least one byte is read.

This method may be invoked at any time. If another thread has already initiated a read operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

**Parameters:**

`dst - The buffer into which bytes are to be transferred`

**Returns:**

`The number of bytes read, possibly zero, or -1 if the channel has reached end-of-stream`

**Throws:**

NonReadableChannelException - If this channel was not opened for reading

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the read operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupt status

IOException - If some other I/O error occurs



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.